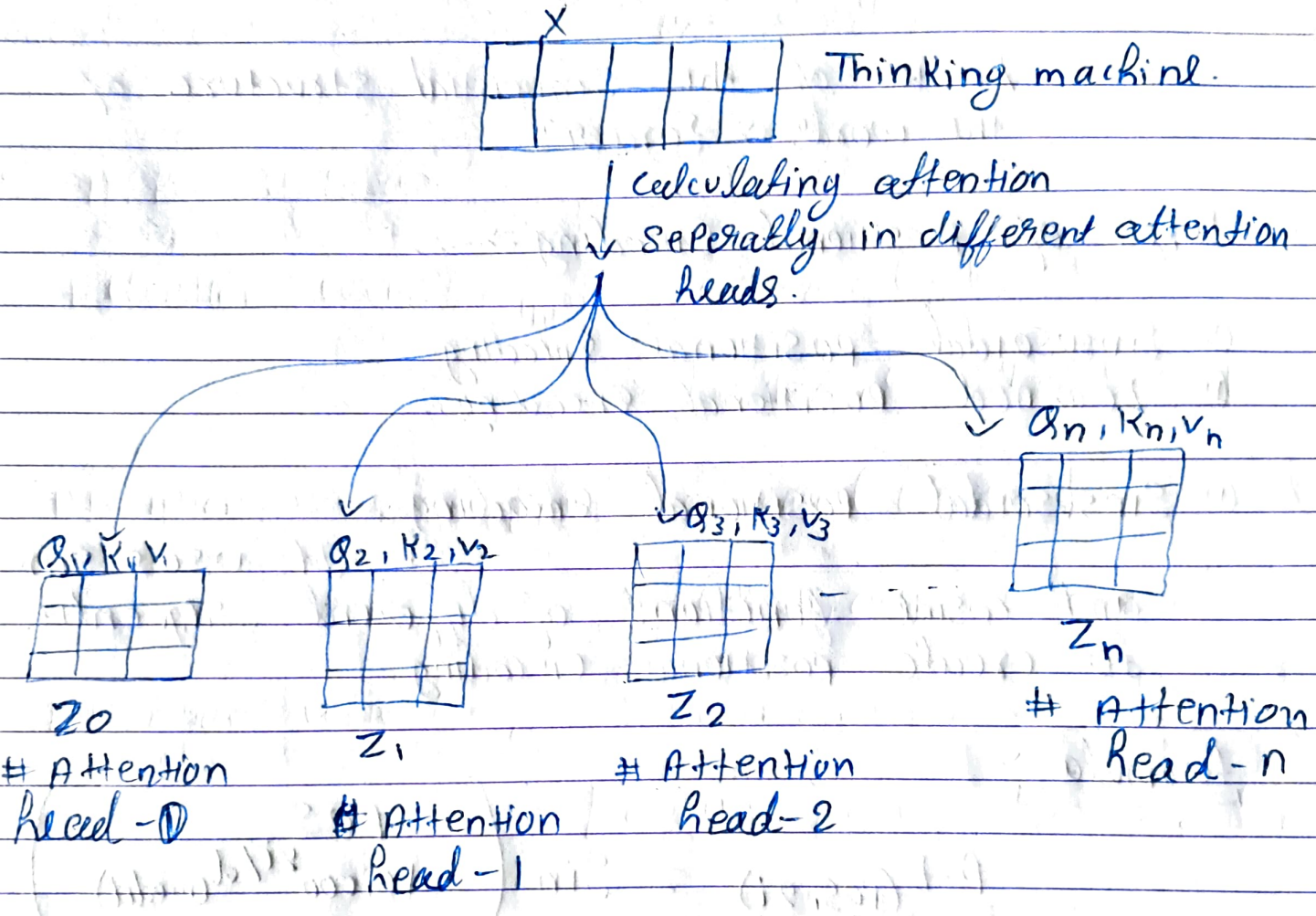
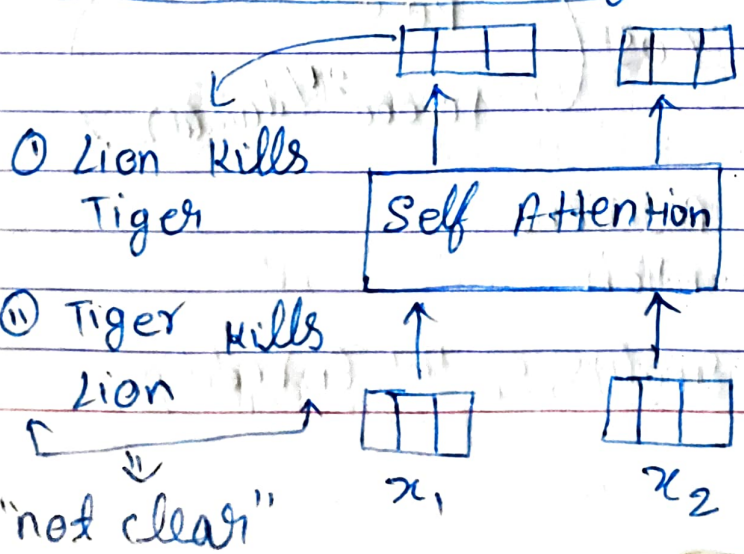
Multi Head Attention.

It expands models ability to focus on different position of tokens.



Positional Encoding. ⇒ Representing order of sequence.



Advantage:- In the transformer the words Tokens are process parallaly

⇓

drawback.

⇓

Lack of the sequential structure of the words ⇒ {order's}

Types of positional Encoding ⇒

① Sinsusoidal positional Encoding
② Learned positional Encoding.

① Sinsusoidal positional Encoding ⇒

It uses sine and cosine functions of different frequenties to create positional encoding.

Formula ⇒

$$P.E_{(pos,\,2i)} = \sin\left(\frac{pos}{10{,}000^{2i/d_{(model)}}}\right)$$

$$P.E_{(pos,\,2i+1)} = \cos\left(\frac{pos}{10{,}000^{2i/d_{(model)}}}\right)$$

⇒ where, pos is the position
i is the dimention
$d_{(model)}$ is the dimensionality of the Embeddings.

Eg → "The cat Sat"

The → [0.1 0.2 0.3 0.4]

Cat → [0.5 0.6 0.7 0.8]

Sat → [0.9 1.0 1.1 1.2]

} → Embedding vectors.

→ $P.E(0,0) = \sin\left(\frac{0}{10{,}000^{0/4}}\right) = 0$

$P.E(0,1) = \cos\left(\frac{0}{10{,}000^{0/4}}\right) = 1$

$P.E(0,2) = \sin\left(\frac{0}{10{,}000^{2/4}}\right) = 0$

$P.E(0,3) = \cos\left(\frac{0}{10{,}000^{3/4}}\right) = 1$

$P.E \Rightarrow [0\ 1\ 0\ 1]$

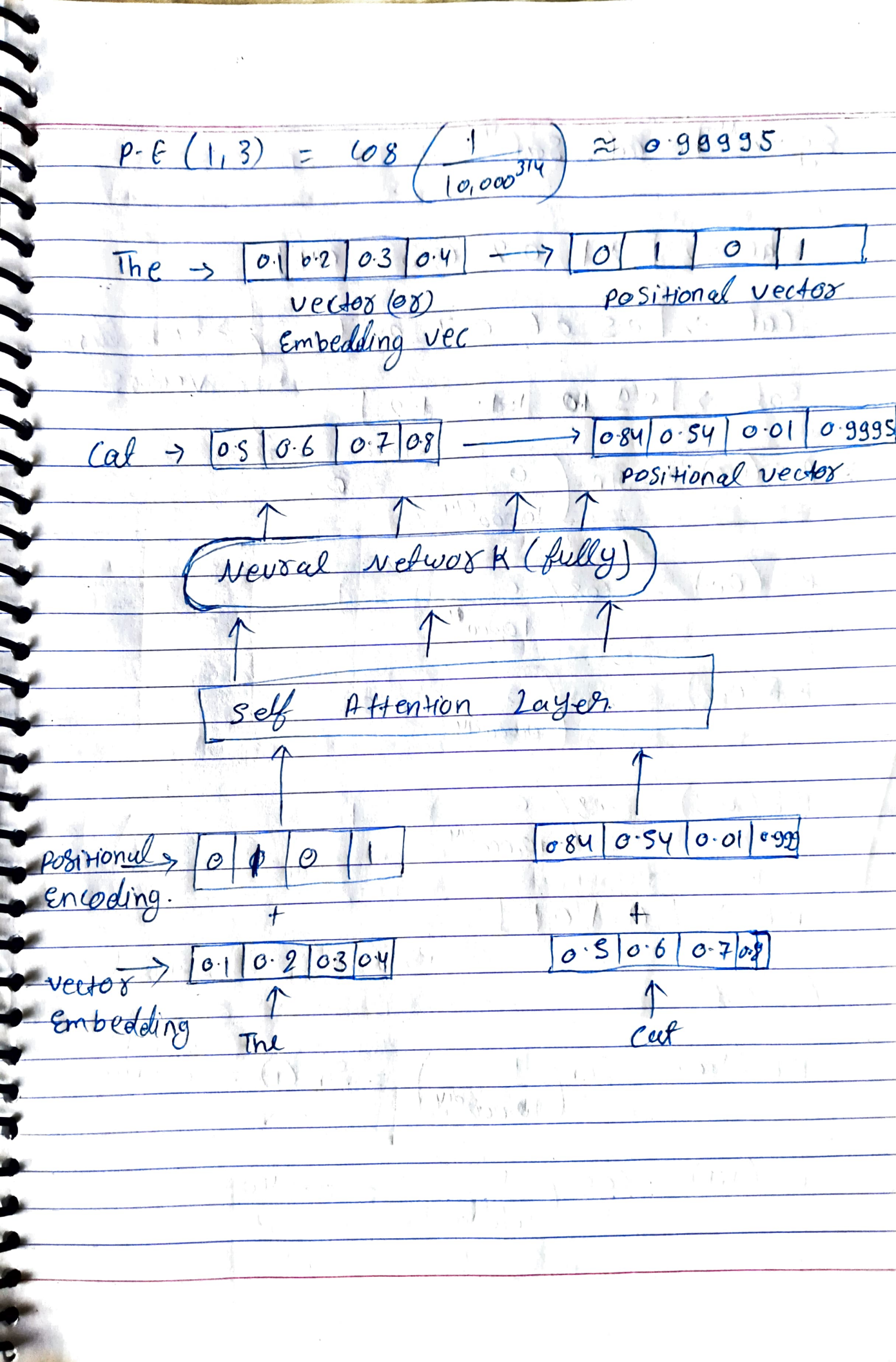
for "Cat"

$P.E(1,0) = \sin\left(\frac{1}{10{,}000^{0/4}}\right) = \sin(1) = 0.8415$

$P.E(1,1) = \cos\left(\frac{1}{10{,}000^{2/4}}\right) = 0.5403$

$P.E(1,2) = \sin\left(\frac{1}{10{,}000^{2/4}}\right) = 0.01$

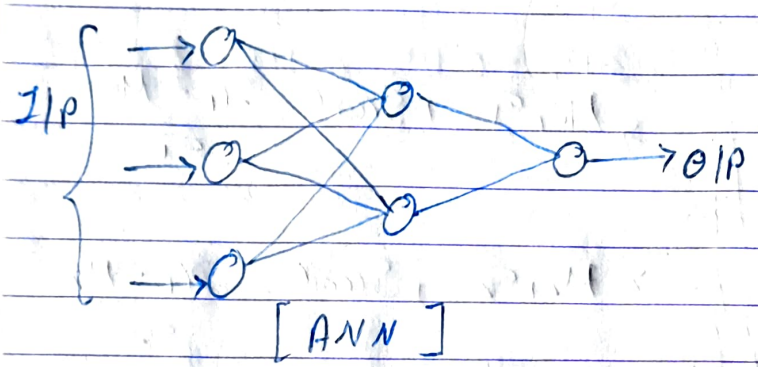
$$P.E(1,3) = \cos\left(\frac{1}{10{,}000^{3/4}}\right) \approx 0.99995$$



Layer Normalization In Transformer.

Normalization → Batch Normalization
Normalization → Layer Normalization.

⇒ Batch Normalization.



features.

f_1	f_2	f_3
House size	Room No	Price (L)
1200	2	45
1500	3	70
900	3.5	80

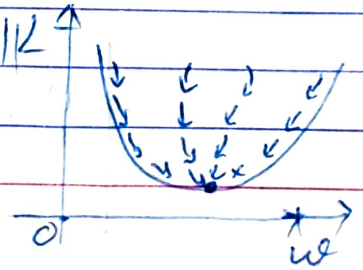
Normalization: Standard scaling. ⇒ Z-Score ⇒ $\frac{x_i - \mu}{\sigma}$

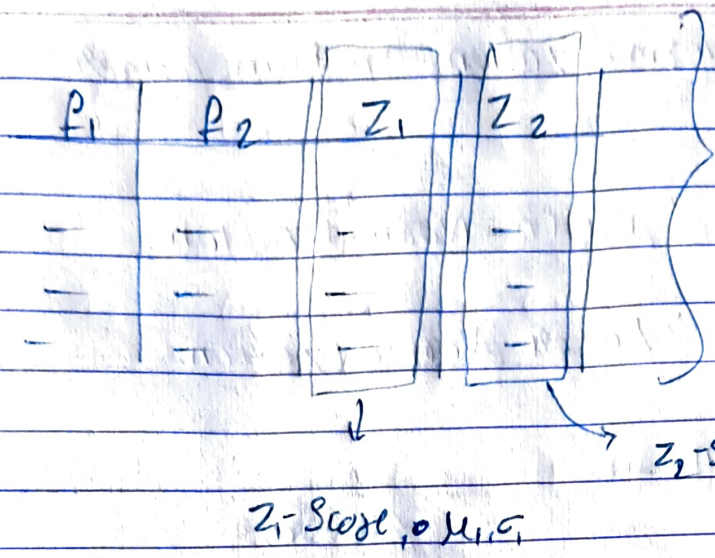
→ $\boxed{\mu = 0, \sigma = 1}$ → for every column.

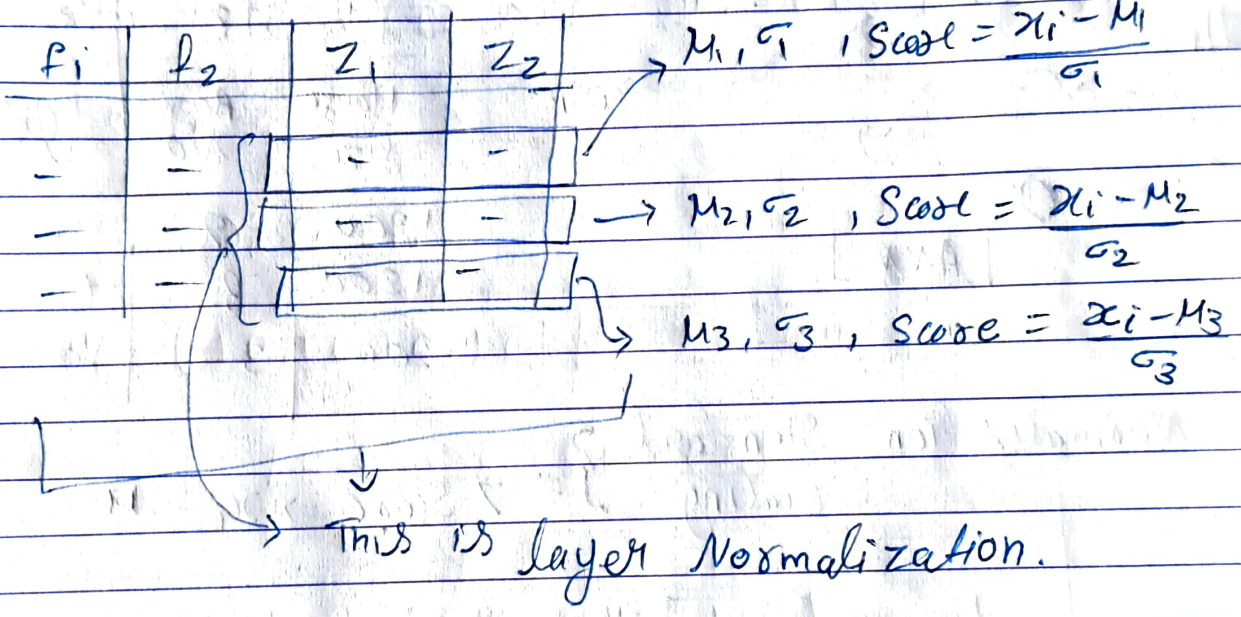
Advantage ⇒

① Improved Training Stability.

② Faster Convergence







⇒ Learnable Parameter.

β, γ

$$z_1 = \sigma\left(\left[w_1^T X + b_1\right]\right)$$

$$y = \gamma\left[\frac{z_1 - \mu_1}{\sigma_1}\right] + \beta$$

→ Scale & Shift Parameter

1) "cat" = [2.0, 4.0, 6.0, 8.0] → Vector

2) Parameters ⇒ $\gamma = [1.0, 1.0, 1.0, 1.0]$ → learned scall.

$\beta = [0.0, 0.0, 0.0, 0.0]$ → shift

i) compute the mean.

$$\mu = \frac{1}{4}(2.0 + 4.0 + 6.0 + 8.0) = \frac{20.0}{4} = 5.0$$

ii) compute the variance (σ^2)

$$\sigma^2 = \frac{1}{4}\left[(2.0-5.0)^2 + (4.0-5.0)^2 + (6.0-5.0)^2 + (8.0-5.0)^2\right]$$

$$\sigma = 5.0$$

iii) Normalize the inputs.

$$\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \varepsilon}} \qquad \varepsilon = 1 \times e^{-5} \Rightarrow \text{Avoid division by 0}$$

$$\sqrt{\sigma^2 + \varepsilon} = \sqrt{5.0 + 1e^{-5}} = \sqrt{5.001} \approx 2.236$$

$$\hat{x}_1 = \frac{2.0 - 5.0}{2.236} \approx -1.34$$

$$\hat{x}_2 = \frac{4.0 - 5.0}{2.236} \approx -0.45$$

$$\hat{x}_3 = \frac{6.0 - 5.0}{2.236} \approx 0.45$$

$$\hat{x}_4 = \frac{8.0 - 5.0}{2.236} \approx 1.34$$

Normalized vector $\Rightarrow [-1.34, -0.45, 0.45, 1.34]$

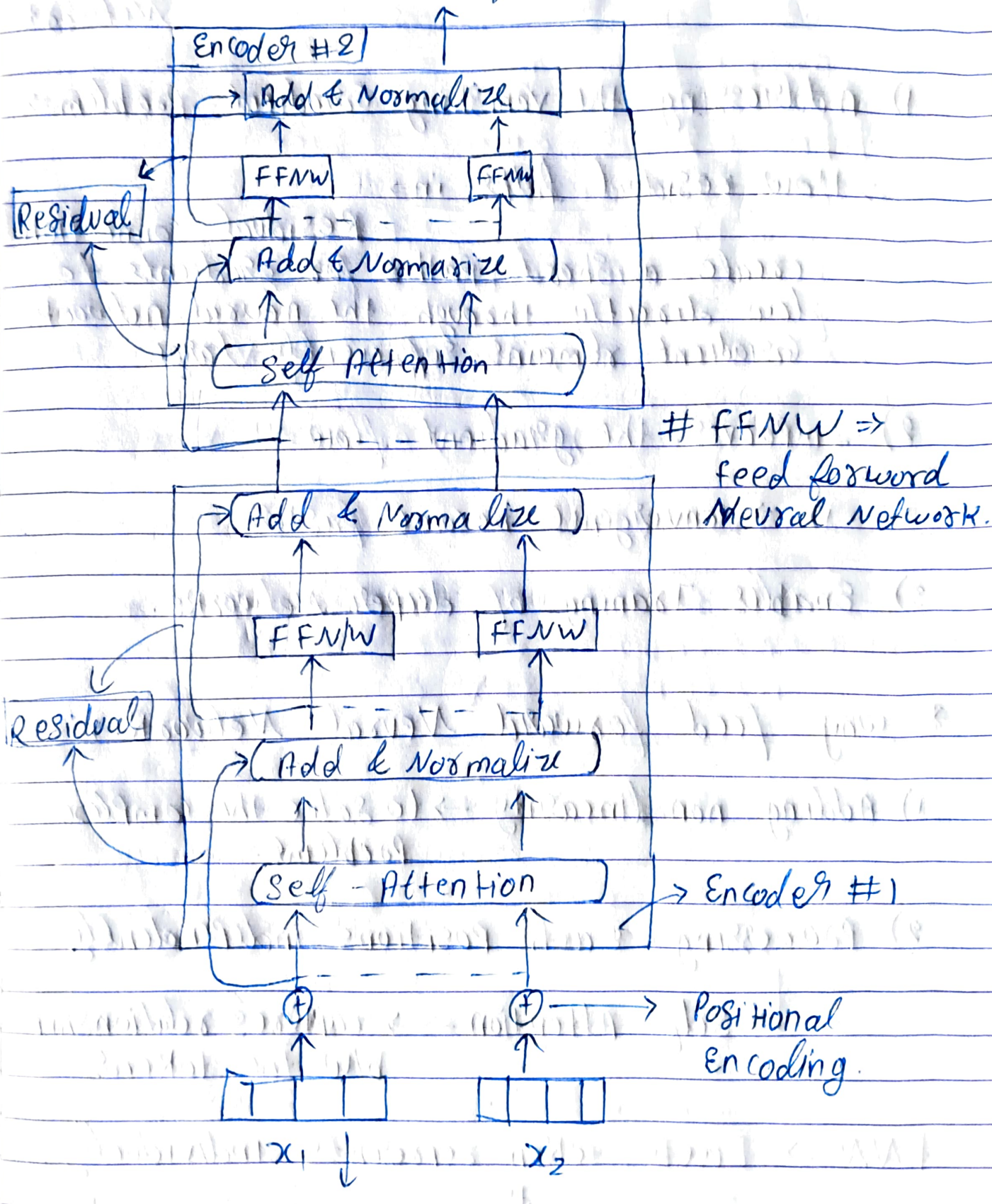
4) Scale & Shift $\Rightarrow$

$$y_i = \gamma_i \hat{x}_i + \beta_i$$

$$y_i = ([1.0, 1.0, 1.0, 1.0] \cdot [-1.34, -0.45, 0.45, 1.34]) + [0.0, 0.0, 0.0, 0.0]$$

$$y_i = [-1.35, -0.45, 0.45, 1.34]$$

Encoder transformer Architecture



FFNW => feed forward Neural Network.

Encoded - vector

* Residual connection ⇒ Skip connection Neural Network.

1) Addressing the vanishing Gradient problem ⇒

⇒ How residual helps in ⇒

Residual connection create a short paths for gradients to flow directly through the neural network Gradient remains sufficiently large.

2) Improves the gradient flow ⇒

i) → Convergence become faster.

3) Enables Traning of deeper Networks.

* why feed forward Neural Network ⇒

1) Adding non linearity → to solve the complex problems.

2) Processing each positions independently.

↳ self-Attention → captures relationship b/w the tokens.

FNN ⇒ Each token represents individually.

⇓

ANN Transforming these representations further, and allows the model to learn richer Representation.

3) FFN → Adds Depth to the model

↳ Depths ↑↑↑ → more learning from the data.

Decoders in transformers

The transformer decoder is responsible for generating the output sequence one token at a time, using the encoder's output and the previously generated tokens.

⇒ 3 main components.

(i) masked multi-head self attention
(ii) multi-head attention (encoder-decoder Attention.
(iii) Feed forward neural Network.

(i) masked multi-head self attention:

- there are mainly three component that we have to discuss.

  (i) Linear projection Q, K, V
  (ii) Scaled Dot Product Attention
  (iii) mask Application → look Ahead mask
  → Padding mask.

① Input Embedding and Positional Encoding.

Output Embeddings:

[[0.1, 0.2, 0.3, 0.4], → Step-1
[0.5, 0.6, 0.7, 0.8],
[0.9, 1.0, 1.1, 1.2], + P.E ⇒ 0
[0.0, 0.0, 0.0, 0.0]]

Step-2: Linear projection for Q, K, V

Create query (Q), key (K) and value (V) vectors.

$W_Q = W_K = W_V = I$

Q = output Embedding $* W_Q$ = output Embedding.
K = output Embedding $* W_K$ = output Embedding.
V = output Embedding $* W_V$ = output Embedding

Step-3: Scaled Dot Product Attention Calculation.

$$\text{Score} = \frac{Q \times K^T}{\sqrt{d_K}} = \frac{Q K^T}{2} \quad \because \sqrt{d_K} = \sqrt{4} = 2$$

$$K_T = \begin{bmatrix} 0.1 \\ 0.2 \\ 0.3 \\ 0.4 \end{bmatrix}$$

$Q_1 = [0.1, 0.2, 0.3, 0.4]$

$\Rightarrow$ Score $\Rightarrow \frac{Q K^T}{\sqrt{d_k}}$ =

$Q_1 \Rightarrow [0.1, 0.2, 0.3, 0.4]$ $\cdot$ $\begin{bmatrix} 0.1 \\ 0.2 \\ 0.3 \\ 0.4 \end{bmatrix}$ $\begin{matrix} K_1 \\ \end{matrix}$ $\cdot$ $\begin{bmatrix} 0.5 \\ 0.6 \\ 0.7 \\ 0.8 \end{bmatrix}$ K_2 $\cdot$ $\begin{bmatrix} 0.9 \\ 1.0 \\ 1.1 \\ 1.2 \end{bmatrix}$ K_3 $\cdot$ $\begin{bmatrix} 0.0 \\ 0.0 \\ 0.0 \\ 0.0 \end{bmatrix}$ K_4

Perform the dot product with all the K's and then it calculate the Q_1 and do all this process for the other queries as well.

the Scores of all Q's are:-

Score: $[[0.3, 0.7, 1.1, 0.0], \rightarrow Q_1$
$[0.7, 1.3, 1.9, 0.0], \rightarrow Q_2$
$[1.1, 1.9, 2.7, 0.0], \rightarrow Q_3$
$[0.0, 0.0, 0.0, 0.0]] \rightarrow Q_4$

masked Application:- It helps to manage the structure of the sequences being processed and ensures the models behaves correctly during training and Inferencing.

Reasons: ① To handle sequences of different length in batch.

2) To ensure that padding tokens, which are added to make sequences of uniform length, do not affect the model prediction.

Eg:- Sequence-1 ⇒ [1, 2, 3]

Sequence-2 ⇒ [4, 5, 0] → 0 is Padding token
↳ influence the Attention mechanism.

when the vector is large then the zero Padding leads to Incorrect or biased prediction.

to solve this we use the masking to Ignore the zero padding.

masking → Padding mask
→ Look Ahead mask

→ if we take Sq-1 → padding mask is: [1, 1, 1]

for Sq-2 → Padding mask is: [1, 1, 0]
↳ represent zero Padding.

(ii) Look Ahead masking → This is used to maintain the Auto regressive property.

① To ensure that each position in the decoder

output sequence can only attend to the previous position, but no future position when we use the Look Ahead mask.

(ii) Sequence-to-Sequence tasks for language modeling, translation.

Eg ⇒ [4, 5, 0] → [1, 1, 0] → ZeroPadding.

Convert 1D to 2D mask.
For each token in the sequence the mask should indicate which token it can attend to.

Token 1 attend to token 1, 2 ←	1	1	0
Token 2 attend to token 1, 2 ←	1	1	0
	0	0	0

Token 3 attend to token 1, 2

{based on the attention mechanism.}

(iii) Look Ahead mask, Decoder output.

[[1, 0, 0],
 [1, 1, 0],
 [1, 1, 1]] → eg.

① Combine Padding And Looking Ahead mask.

⇒ Element wise multiplication of 2 mask.

⇒ Combained mask ⇒ [[1, 0, 0]
[1, 1, 0]
[0, 0, 0]]

⇒ mask

scores : [[0.3, 0.7, 1.1, 0.0],
[0.7, 1.9, 3.1, 0.0],
[1.1, 3.1, 5.1, 0.0],
[0.0, 0.0, 0.0, 0.0]]

Look Ahead mask ⇒ [[1 0 0 0],
[1 1 0 0],
[1 1 1 0],
[1 1 1 1]]

Padding mask ⇒ [[1, 1, 1, 0],
[1, 1, 1, 0],
[1, 1, 1, 0]
[0, 0, 0, 0]]

Combained mask ⇒ Look Ahead * Padding mask.

⇒ [[1x1, 0x1, 0x1, 0x0],
[1x1, 1x1, 0x1, 0x0],
[1x1, 1x1, 1x1, 0x0],
[1x1, 1x1, 1x1, 1x0]]

⇒ element wise multiplication.

→ [[1,0,0,0],
[1,1,0,0],
[1,1,1,0],
[1,1,1,0]]

→ [[1, -∞, -∞, -∞],
[1, 1, -∞, -∞],
[1, 1, 1, -∞],
[1, 1, 1, -∞]].

Conver the $0 \rightarrow -\infty$

masked score →

[[0.3, -∞, -∞, -∞],
[0.7, 0.9, -∞, -∞],
[1.1, 3.1, 5.1, -∞],
[0.0, 0.0, 0.0, -∞]].

↓

Zero out the influence when the soft-max is applied.

→ Softmax

Soft-score = Softmax (masked score)

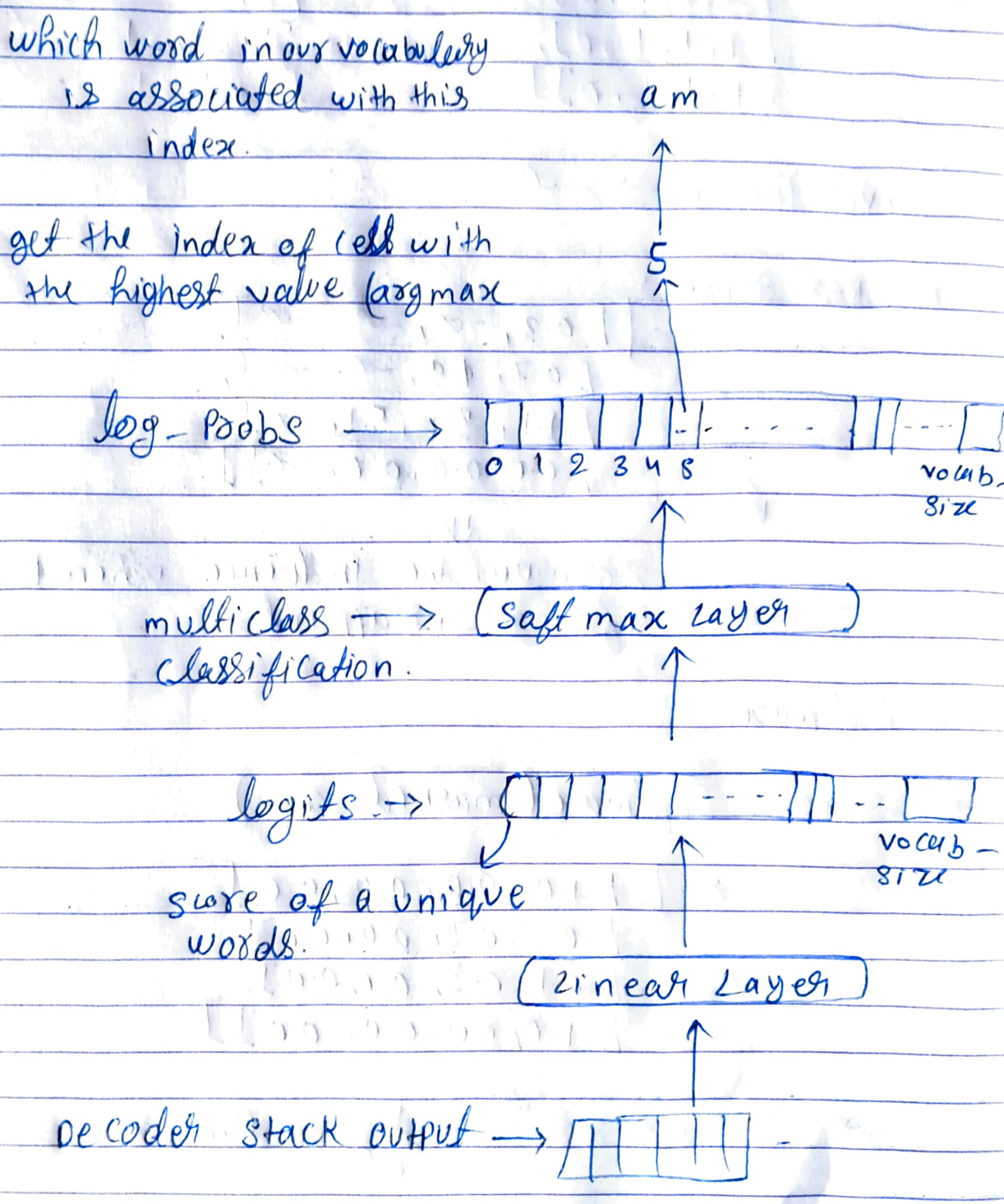
→ [[1.0, 0.0, 0.0, 0.0],
[0.3, 0.7, 0.0, 0.0],
[0.1, 0.3, 0.6, 0.0],
[1.0, 0.0, 0.0, 0.0]]

→ because $e^{-\infty} = \frac{1}{e^{\infty}} \approx 0.0$

→ weighted sum of values:

{Attention O/P = Soft-score x V}.

Final Linear And Softmax Layer.



Linear => The Linear layer is a simple fully connected neural nlw that projects the vector produced by the stack of Decoder.

model ⇒ 10,000 vocab-size → logits vector

⇓

10,000 cell size.

⑥ Softmax layer turns those scores into probabilities (all add upto 1.0). Then the cell with the highest probability is chosen, and the word associated with it is produced as the O/P ⇒ with specific time stamp.